

Allocator-Aware Rust

NDC TechTown

Kongsberg - September 2026

Victor Ciura

Principal Architect
Rust Tooling @ Microsoft

The deal

What you walk out with

- A different **suspect**: why memory *placement* explains stalls you have been blaming on the CPU.
- A vocabulary for **characterizing a subsystem** before you touch its allocator.
- A **diagnostic workflow** you can run on the allocator you already ship.
- A clear picture of what Rust can express **today**, and what is genuinely coming.

A Story in Three Parts

Part 1 - Your heap is the crime scene

A primer, a challenge, some questions about your memory - and a massive case study.

Part 2 - Where local allocation pays off

Arenas, wink-out, contention, NUMA, and the benchmark evidence.

Part 3 - Are we allocator-aware yet?

The Rust `Allocator` contract, the MVP, and what works on stable today.

Part 1

Your heap is the crime scene

One-liner

```
use mimalloc::MiMalloc;  
  
#[global_allocator]  
static GLOBAL: MiMalloc = MiMalloc;
```

Q# benchmark	Change in elapsed time
Array literal evaluation	-39.5%
Standard library	-28.4%
Large file parity evaluation	-1.8%

A low-effort experiment, that can pay off big. Not a guaranteed win, consult your memory doctor before applying.

The win is real. Do you know *why* it worked?

Contention? Fragmentation? Per-call cost? Placement?

- Four completely different mechanisms. **Different ceilings. Different next steps.**
- If you cannot say which one you bought, you cannot say what is left on the table.

First clue: don't stop at how fast. Ask where.

The thesis

1. **Understanding how allocators work is essential** - even if `Global` is the only allocator you ever use. Your default allocator makes a placement decision on every allocation you write.
2. **Stop blaming the CPU: your heap is the crime scene.** Memory access patterns can dominate performance in long-running systems.
3. **Not understanding and choosing the right allocation strategy means leaving performance on the table.** The right boundary can matter even after allocation has finished.

A primer, in one slide

An allocator manages a possibly non-contiguous memory region, handing out suitable storage on demand.

What it must do

- Hand out `size` bytes at `align` alignment
- Keep independently allocated **live blocks disjoint**
- Take it back (maybe)

What it is silently deciding

- **Which** address you get
- **Near what** other data you get it
- **When** that address is recycled
- **Who else** contends for the metadata

The left column is the contract. The right column is your performance - and you get it whether or not you ever name an allocator.

C++ and Rust are unusual here

Runtimes with a moving GC

- Bump-allocate, then a **moving GC** compacts
- The runtime periodically **re-packs** the live set
- You pay in pauses and in headroom

C++ and Rust

- Heavily **stack-friendly**
- **Deterministic** reclamation tied to scope
- Nobody ever comes along and tidies your heap

Without automatic compaction, locality becomes an explicit design choice.

Two axes, four quadrants

	General-purpose	Special-purpose
Global	<code>malloc</code> / <code>System</code> tcmalloc, jemalloc, mimalloc, sncmalloc	tracing / counting wrappers guarded or tagged heaps
Local	a general-purpose allocator over an arena a synchronized multipool	monotonic (bump) thread-confined multipool , stack , <code>alloca</code>

- **General-purpose**: works *reasonably* well for *all* use cases. Must satisfy every requirement.
- **Special-purpose**: works *especially* well for *some* use cases. Need not satisfy all of them.

The global allocator menu

Allocator family	Mechanisms to investigate
System	Platform allocation, reuse and page-retention policy
tcmalloc	Per-CPU or per-thread caches, shared backing storage
jemalloc	Thread caches, shared arenas, size classes
mimalloc	Page-local free-list sharding
snmalloc	Batched remote deallocation messages

A global swap can help. It does not express your intent.

The ordinary global allocation interface does not tell the allocator:

- that these 40,000 nodes form **one graph**, walked together, discarded together
- that a subsystem is **done** and its whole region can be reclaimed at once
- which allocations should be kept **away from everybody else's**

Encode grouping in your **data layout**, or supply an explicit **allocation boundary**.

Placement heuristics can help. An explicit boundary communicates intent.

What everyone measures: Fragmentation

The story we all know

- Allocate, free, allocate, free
- Holes appear between live blocks
- Holes too small to reuse
- RSS climbs; you "leak" without leaking

Why we fixate on it

- We like reach for a **single proxy** (RSS)
- It is easy to **explain**
- Modern allocators **mitigate a lot of it** - size classes, bins, page reuse

Fragmentation is real, and it is the part of the problem the industry has spent the most effort mitigating for us. It is not solved, and it is not the only indicator you should care about.

No holes. Still poor locality.

Fragmentation

Memory is **holey**.

Gaps you cannot reuse.

A **space** problem.

Clue: RSS vs live bytes.

Fragmented



Diffused



Diffusion

Memory can be **compact** -

and everybody's data is **intermixed**.

A **time** problem.

Clues: cache misses, TLB misses, stalls.

A perfectly compact heap can still have poor locality. No holes does not mean related data is together.

Two costs regimes, sometimes in the same program

Allocation-cost dominated

Build up, use briefly, tear down.

The win is **faster allocate / deallocate**.

Bump allocation is a pointer increment.

Deallocation can be a **no-op**.

Access-cost dominated

Return to the same data, again and again.

The win is **improved memory access**.

Locality, cache lines, TLB, prefetch.

Diffusion is the enemy.

The cost of accessing memory can outweigh the cost of obtaining it.

Characterizing a subsystem

	Ask about this subsystem	Useful evidence
Fragmentability	Can related live blocks be intermixed with other work?	The live allocation graph
Density	How much work allocates or frees?	Allocation calls per operation
Variation	How much do requested sizes vary?	A size histogram
Locality	Which data is revisited together?	Access pattern and working set
Utilization	How much allocation is churn?	Peak live / cumulative bytes
Contention	Who allocates and frees concurrently?	Threads, handoffs, shared metadata

Choose a subsystem and a time window. Use these indicators to choose an experiment - including "change nothing".

A Case Study

100 TB, without leaving the global allocator

The setting

```
pub struct CacheEntry {  
    timestamp: UnixTimeStamp,  
    pub inception: Instant,  
    pub ttl: Ttl,  
    pub hits: u32,  
    pub answers: Vec<Record>,  
    pub authority: Vec<Record>,  
    pub additional: Vec<Record>,  
    pub errors: Vec<ExtendedError>,  
    // ...  
}
```

- 250+ billion entries. **One wasted byte = 250 GB**. Written once, then only read.
- Already on **jemalloc**: threads allocate from per-thread caches backed by a small number of arenas, so the fast path avoids central contention; blocks are binned into **size classes**, so a freed entry's slot can serve the next request in that class.

An immutable entry and an evicting cache have different allocation lifecycles.

Fix: stop paying for capacity you will never use

Before

```
pub answers: Vec<Record>,  
// ptr + len + capacity = 24 bytes  
// plus over-allocated heap slack
```

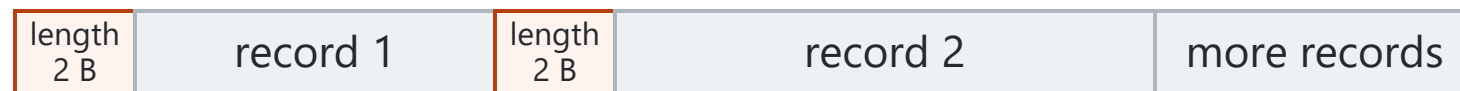
After

```
pub answers: Box<[Record]>,  
// ptr + len = 16 bytes  
// no spare element capacity
```

- `Vec` carries `capacity` so it can grow. This entry never grows.
- 8 fields x 8 bytes = 64 bytes per entry, plus the slack `Vec` reserves for growth.
- At 250 billion entries: over 15 terabytes.

Fix: contiguous layout, without a local allocator

records: Box<[u8]>



One global allocation; field widths are schematic.

- Eliminates the per-variant enum overhead **and** the separately boxed record payloads.
- Data is **packed contiguously** - locality restored, and most records can be **copied straight out** into the response.
- The scratch buffer **persists across insertions** and rarely reallocates.
- The cost, named: records are **no longer randomly indexable** - you iterate. Counts per entry are small, so they took the trade.

Insert throughput, from this change alone	+13%
Lookup latency, from less serialization + better locality	-5%

The results

Metric	Before	After	Change
Per-entry net footprint (<i>benchmark</i>)	953 B	420 B	-56%
Per-entry allocated bytes (<i>benchmark</i>)	1.1 KB	461 B	-58%
Cache insert throughput (<i>benchmark</i>)	625,000/s	893,000/s	+43%
Cache lookup latency (<i>benchmark</i>)	828 ns	670 ns	-19%
p99 resident memory per instance (<i>production</i>)	9.3 GB	5.3 GB	-43%
p90 resident memory per instance (<i>production</i>)	6.5 GB	3.8 GB	-42%

~100 TB freed across Big Pineapple's fleet. These changes introduced no local allocator.

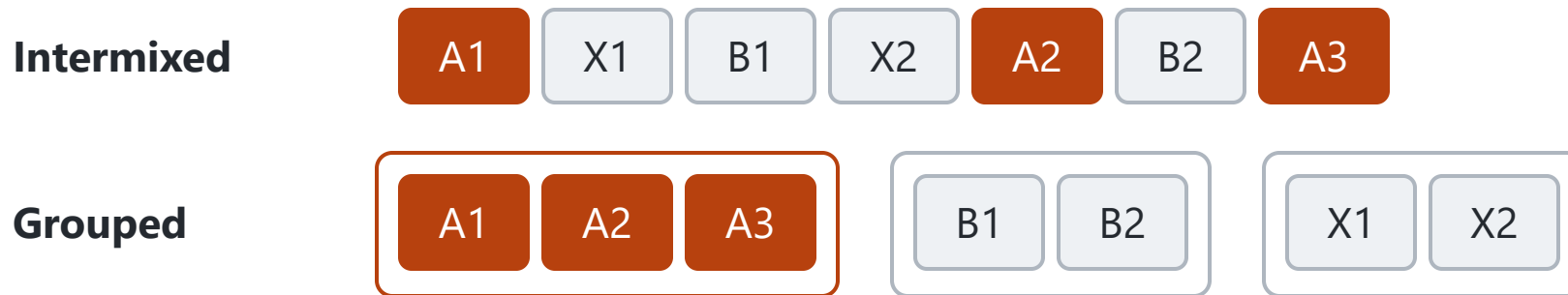
Next question: what if independently allocated objects must remain?

Part 2

Where local allocation pays off

A local allocator, in one picture

- A **sub-region** of memory - an *arena* - handed to a subsystem.
- It may live **much less** than the program.
- It is **supplied**, not looked up globally.
- A managed arena can reclaim its **backing chunks as a unit**, subject to its validity contract.



Same objects. Same traversal. Different grouping in memory.

Monotonic (bump) allocation

Policy pseudocode, not an implementation

```
allocate(layout):  
    start = align_up_checked(cursor,  
                             layout.alignment)  
    end = checked_add(start, layout.size)  
  
    if arithmetic failed or end > chunk_end:  
        return allocate_from_new_chunk(layout)  
  
    cursor = end  
    return block(start, layout.size)  
  
deallocate(block):  
    do nothing
```

Properties

- Fast-path allocation: **align + add**. No free-list search or per-object metadata.
- Deallocation: **nothing at all**.
- **Reset/release or destruction** reclaims collectively; reset may retain chunks.
- Allocations are **adjacent within a chunk** (modulo alignment padding).

Costs

- Never reuses individually freed memory.
- Needs a bounded, known-ish working set.

Stack allocators: the arena you already have

C++ - arena over a stack buffer

```
void handle(Request const& r) {
    std::byte buf[16 * 1024]; // stack
    std::pmr::monotonic_buffer_resource
        arena{buf, sizeof buf};
    std::pmr::vector<Item> items{&arena};
    // Bump while buf lasts; upstream on overflow.
    parse(r, items);
    respond(items);
} // storage ends with this frame
```

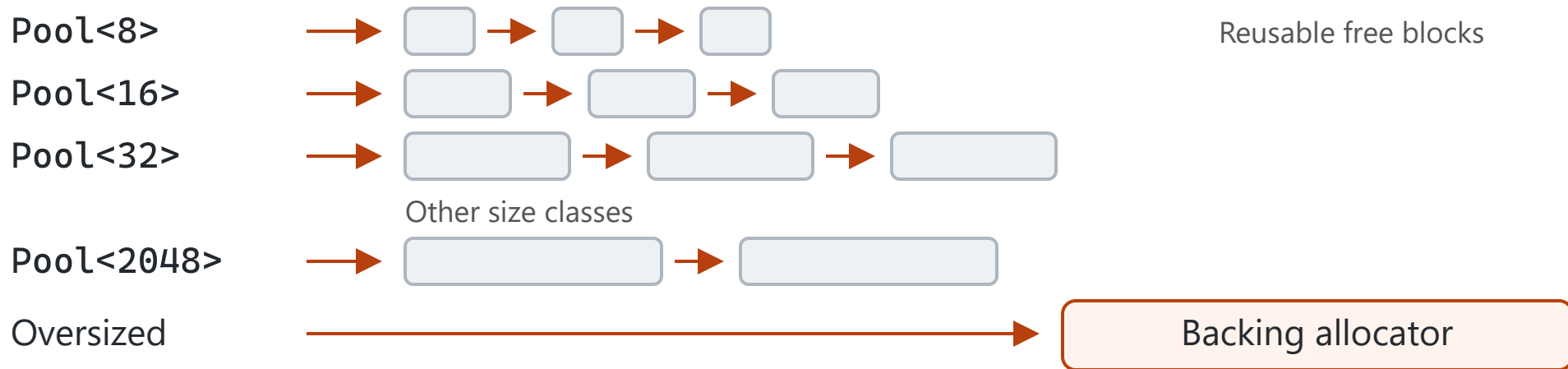
Rust - inline storage, with an overflow policy

```
fn handle(r: &Request) {
    // spills to the heap only past 32
    let mut items: SmallVec<[Item; 32]>
        = SmallVec::new();
    parse(r, &mut items);
    respond(&items);
} // drops here; returning transfers ownership

// fixed capacity, never spills:
let mut buf: ArrayVec<u8, 1024>
    = ArrayVec::new();
```

Choose overflow behavior: **spill**, **fail**, or **reject**. Returning transfers ownership: inline elements may move; a spilled heap buffer does not.

Multipool: size classes, locally



- Each pool is a **free list of one size**. Its allocate/free fast paths are $O(1)$; refills cost extra.
- **Reuses** freed memory, unlike monotonic - so it survives low **U** (churn).
- Still **local**: blocks come from one arena, so the subsystem stays clustered.

Which allocator? A decision path

If your subsystem...	...reach for
builds up, is used, then torn down whole	monotonic (bump)
is small, bounded, and frame-scoped	stack / inline storage
churns for hours with many small, same-ish sizes	multipool
churns across bounded, pooled size classes	multipool , optionally over monotonic
churns with sizes above the pooling threshold	route to a reclaiming allocator
is touched by a thread pool	subsystem/task-owned regions; coordinate shared use
can skip cleanup and safely reclaim its region	monotonic + wink-out
has low F (one contiguous allocation)	no diffusion to prevent ; assess other needs

What Rust can express: `new_in`

Rust (stable / allocator-api2)

```
use allocator_api2::{boxed::Box, vec::Vec};
use bumpalo::Bump;

let arena = Bump::new();

let b: Box<u32, &Bump>
    = Box::new_in(42, &arena);

let mut v: Vec<u32, &Bump>
    = Vec::with_capacity_in(1024, &arena);

// the allocator is part of the TYPE
fn takes(v: Vec<u32, &Bump>) { /* ... */ }
```

C++17 (pmr)

```
#include <memory_resource>

std::byte buf[16 * 1024];
std::pmr::monotonic_buffer_resource
    arena{buf, sizeof buf};

std::pmr::vector<int> v{&arena};

// the allocator is erased behind
// a memory_resource*
void takes(std::pmr::vector<int> v);
```

PMR erases the resource type; Rust here uses a defaulted type parameter: `Vec<T, A = Global>`.

Using an arena today: `bumpalo`

```
use bumpalo::{Bump, collections::Vec as BumpVec};

struct Node<'a> { id: u32, name: &'a str }

let arena = Bump::new();
{
    let mut nodes = BumpVec::with_capacity_in(40_000, &arena);
    for id in 0..40_000 {
        let name = bumpalo::format!(in &arena, "n{}", id)
            .into_bump_str();
        nodes.push(Node { id, name });
    }
    process(&nodes);
} // owning collection finishes before its arena
drop(arena); // release backing chunks
```

- Stable Rust; bumpalo's `collections` feature. The arena is `&arena` - a borrow.
- **That ampersand is about to become the most important character in this talk.**

One class of bug, rejected by the type system

C++ - compiles, leaves dangling storage

```
std::pmr::vector<int> make() {  
    std::byte buf[1024];  
    std::pmr::monotonic_buffer_resource  
        arena{buf, sizeof buf};  
    std::pmr::vector<int> v{&arena};  
    v.push_back(42);          // storage in buf  
    return v;  
} // buf and arena are gone.  
    // v still points at both. Silent UB.
```

Rust - does not compile

```
fn make() → BumpVec<'static, i32> {  
    let arena = Bump::new();  
    let mut v = BumpVec::new_in(&arena);  
    v.push(1);  
    v  
}  
// error[E0515]: cannot return value  
// referencing local variable `arena`
```

"An object must not outlive its allocator" is a lifetime rule.
For a borrowed allocator, Rust enforces it at compile time.

An outer arena is not enough

The collection moved; the bytes did not

```
let mut names = BumpVec::new_in(&arena);
names.push(format!("n{}", 7));

// Vec buffer: arena
// String buffer: Global
```

Construct the payload in the region too

```
let mut names = BumpVec::new_in(&arena);
let name: &str =
    bumpalo::format!(in &arena, "n{}", 7)
        .into_bump_str();
names.push(name);

// Vec buffer and name bytes: arena
```

- A container's allocator choice is **not automatically inherited** by its elements.
- Library authors must expose and propagate the boundary through constructors and helpers.

A checked lifetime is not a checked placement policy.

Memory-safe is not the same as a good policy

One arena for the whole service

```
let arena = Bump::new();
for request in requests {
    let scratch =
        arena.alloc_slice_copy(request);
    consume(scratch);
}
// Scratch storage accumulates until
// this outer arena is released.
```

Reclaim at the phase boundary

```
let mut arena = Bump::new();
for request in requests {
    {
        let scratch =
            arena.alloc_slice_copy(request);
        consume(scratch);
    }
    arena.reset();
}
```

Both compile. Only one matches this scratch buffer's lifetime.

Wink-out

Reclaim an object graph without running its destructors

```
let arena = Bump::new();  
let graph = build_huge_graph(&arena); // arena-only resources; no pinned values  
walk(&graph);  
  
core::mem::forget(graph); // skip this owner's drop traversal  
drop(arena);              // release the chunks
```

- No required cleanup beyond returning memory to **that arena**.
- When a drop traversal is avoided: **$O(\text{objects})$** teardown becomes **$O(\text{chunks})$** .
- `mem::forget` is safe. **Reclaiming storage is a separate obligation.**
- Borrowed-arena APIs prevent later safe access; **pinning/drop contracts still apply.**

Wink-out: the fine print

C++

- Skipping a destructor that owns a resource is a **leak** - or worse
- Nothing in the type system separates "owns only arena memory" from "owns a socket"
- Detection: review, test allocators, static analysis

Skipping cleanup and reclaiming storage are different decisions.
Memory safety does not make skipped resource cleanup correct.

Rust

- **Forgetting is safe.** `mem::forget` alone does not authorize freeing the storage
- Sound arena APIs must prevent later access and uphold **pinning/drop guarantees**
- `Bump::alloc` deliberately **does not** run `Drop` - the trade is explicit in the API

Contention

- Global allocators already use **thread caches and sharding** to reduce contention.
- Async and work-stealing workloads can complicate the picture:
 - a worker mixes allocations from **different subsystems**
 - allocation and freeing may happen on **different threads**
 - cache refills and remote frees may require coordination

A private arena avoids shared allocator metadata on its chunk fast path. Refills and concurrently shared arenas still need coordination.

Local allocators vs. thread-local storage

TLS

- Binds memory to a **thread**
- Needs **affinity**: pin the subsystem to the thread
- Work stealing breaks the assumption
- Hard to hand off, bound, or release

Local allocator

- Binds memory to a **subsystem**
- Group the allocator **with the data**
- Released as a unit when the subsystem dies
- Migration needs **Send** state; concurrent allocation still needs sharding or synchronization

Subsystem grouping need not follow worker identity. Cache warmth and NUMA placement still depend on where the work runs.

The result can outlive the phase (Multitude)

```
use multitude::{Arc, Arena};

let message: Arc<str> = {
    let arena = Arena::new();
    arena.alloc_str_arc("ready for the next stage")
}; // The arena handle is gone. Its chunk is not.

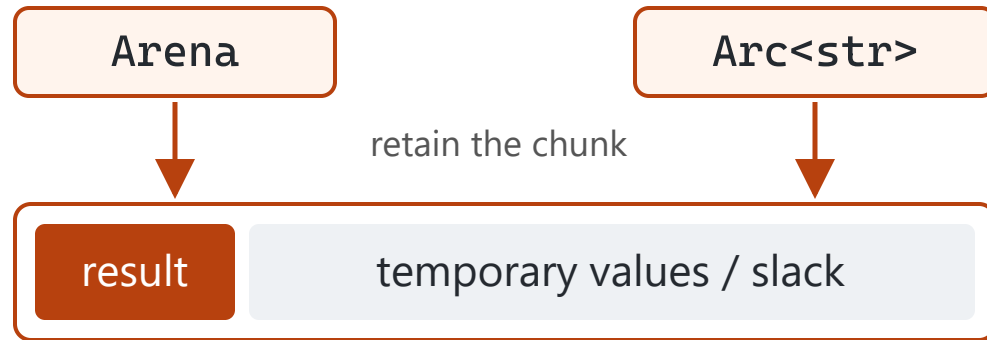
std::thread::spawn(move || println!("{message}"))
    .join().expect("worker panicked");
```

- Arena-bound: `Alloc<'a, T>` borrows the arena.
- Escape-capable: `multitude::Box`, `Rc`, and `Arc` retain their backing chunks.

The result outlives the arena handle, not its storage.

You retain chunks, not just results (Multitude)

During construction



After the arena is dropped



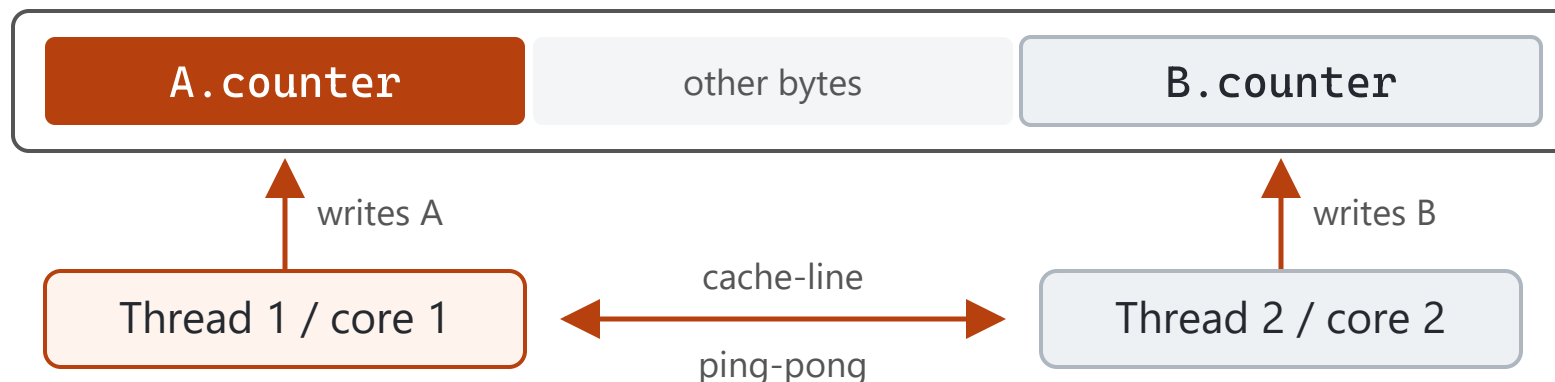
A small live result can keep a much larger chunk allocated.

- A small survivor can retain a **whole chunk**.
- **Value destruction**, chunk reuse, and returning memory upstream are different events.
- No wink-out shortcut: forgetting these owners **leaks their chunk references**.

A checked ownership contract still needs a measured retention policy.

False sharing

One cache line (64 B in this example)



- Diffusion **can cause** false sharing when independently written data shares a cache line.
- Separate arenas prevent **cross-arena** sharing when their backing chunks occupy disjoint cache lines.
- The alternative is per-field and manual: `#[repr(align(64))], alignas( ... )`.

NUMA and affinity

- On Linux and Windows, a page is typically backed on the NUMA node of the core that **first touches** it.
- Thread-local allocation and reuse **can help**, if execution stays on that node.
- Three ways that quietly breaks:
 - `numactl --interleave` (common for databases) **defeats** first-touch deliberately
 - a work-stealing pool moves the task *after* the memory was placed
 - a global free list hands you back memory that some **other node** touched first

An arena can apply a deliberate policy to its **backing pages**. Moving the task does not move those pages.

Work the crime scene

A diagnostic workflow - no allocator engineering required

Count and attribute

1. **Count allocations** per operation - a wrapper over the allocator you actually use.
2. **Attribute** them: `dhat`, `heaptrack`, tagged allocation sites.
3. **Track live bytes vs RSS**. A widening gap is a *hypothesis* - fragmentation, retained pages, or metadata.

Then test the hypothesis

4. **Replay** the same allocation and free history. Placement effects need the history, not just elapsed time.
5. Correlate with **LLC and dTLB misses per operation** (`perf stat`).
6. `perf c2c` for false sharing; `numastat` for where pages were **allocated**.

Flat live data + worsening latency + more misses per operation = placement is worth investigating.
Then change one thing and re-run.

The allocator as an instrument

```
unsafe impl<A: GlobalAlloc> GlobalAlloc for Counting<A> {
    unsafe fn alloc(&self, l: Layout) → *mut u8 {
        self.record(l.size()); unsafe { self.inner.alloc(l) }
    }
    unsafe fn alloc_zeroed(&self, l: Layout) → *mut u8 {
        self.record(l.size()); unsafe { self.inner.alloc_zeroed(l) }
    }
    unsafe fn realloc(&self, p: *mut u8, l: Layout, n: usize) → *mut u8 {
        self.record(n); unsafe { self.inner.realloc(p, l, n) }
    }
    unsafe fn dealloc(&self, p: *mut u8, l: Layout) {
        unsafe { self.inner.dealloc(p, l) }
    }
}
```

Beyond this counter: tracing, controlled exhaustion, leak detection, subsystem budgets.

Placement policies: high-bandwidth, protected, mapped or shared memory.

Requests, not live bytes. Delegate all four calls to the selected allocator; setup in notes.

The usual counter-arguments

"Modern global allocators solved this."

They improve call cost, reuse, locality and contention. Explicit application boundaries still supply information the ordinary global interface does not.

"Virtual dispatch costs too much."

Small in some measured configurations, not universally negligible. Concrete Rust allocator types use static dispatch; measure the trade.

"The testing burden is impracticable."

Allocators are **opt-in**. Borrowing makes outliving the allocator a **compile error**. Allocation policy and skipped cleanup still need testing.

"This is premature optimization."

Choose a representative operation and establish a baseline. Change policy when measurements or placement requirements justify it; otherwise keep the simpler choice.

The Cost

Up-front (infrastructure)

- Plumbing allocators through constructors and sub-objects
- More source code
- Training, review, tooling
- Borne mostly by **library authors**

Incremental (every user)

- A wider interface and a wider **contract**
- Cognitive load at every construction site
- **Costly misuse**: dangling allocators (C++), unbounded monotonic growth
- Borne by **most application developers**

Choose the engineering investment **per subsystem**, not per company.

Part 3

Are we allocator-aware yet?

The trait, as it stands today

```
pub unsafe trait Allocator {  
    // Required  
    fn allocate(&self, layout: Layout)  
        → Result<NonNull<u8>, AllocError>;  
    unsafe fn deallocate(&self, ptr: NonNull<u8>,  
                        layout: Layout);  
  
    // Provided operations:  
    // allocate_zeroed, grow, grow_zeroed, shrink  
}
```

- Two required methods. Everything else defaulted.
- RFC accepted in 2016. **MVP merged September 23, 2026; not yet released stable.**

Why 10 years?

- Many attempts began with **one more API redesign** before stabilization.
- Larger alternatives, including the **Store** API, competed with an MVP approach.
- **Soundness issues** surfaced alongside the design questions.
- The merged 2026 MVP: establish a sound minimum; leave room for compatible extensions.

A deferred capability split: **Deallocator**

```
unsafe trait Deallocator { unsafe fn deallocate(&self, ptr: NonNull<u8>, layout: Layout); }
unsafe trait Allocator: Deallocator {
    fn allocate(&self, layout: Layout) → Result<NonNull<[u8]>, AllocError>;
}
```

For a bump allocator, freeing is a **no-op** - it needs strictly less information than allocating. Collections that only ever free could require just **Deallocator**.

Where it actually stands

Released stable: 1.98.1

Global replacement and arena crates work. Allocator-generic ecosystem collections are available; std's MVP is **not released yet**.

Still experimental

`allocator_ext`: remaining APIs, including `AllocatorClone`, `StaticAllocator`, and `GlobalAllocator`.

Snapshot: September 23, 2026. A standard-library merge does not update crate integrations.

Merged MVP: milestone 1.100.0

`Allocator`, minimal allocator-generic `Box` / `Vec` APIs, and `Global` / `System` implementations.

Further design work

`Deallocator` split, `AllocatorEq`, `MIN_ALIGN`, allocation flags, the `Store` API.

Pin, covariance, and `Box`

`A: 'static`

No short-lived borrowed state in the type.

An owned arena can still **free storage on drop**.

`StaticAllocator`

No implicit invalidation of allocations.

An **unsafe storage-validity promise**, not an immortal allocator value.

- `std::boxed::Box<T, A>` is **covariant** over `A` ; allocator lifetimes can shorten.
- Combined with custom boxed cloning, that let a pinned box acquire a **shorter-lived allocator**.
- `StaticAllocator` protects the contract through those coercions. **Experimental, not the MVP.**

Rust changes the economics

Rust: checked borrowing

- A borrowed region's **lifetime** is in the type
- **Defaulted allocator parameter**; concrete dispatch
- **Send / Sync** express transfer and sharing obligations
- Forgetting is safe; **storage validity** and cleanup still need a design

C++: established integration

- **PMR** standardized in C++17
- A runtime-polymorphic **resource interface**
- **Production experience** and supporting tools
- A standard **stack-backed resource idiom**

Rust can check a borrowed region's lifetime. Choosing and propagating its policy still takes engineering.

What you can do today, on stable

No allocator engineering required

1. **Baseline:** allocation counts and latency.
2. **Global swap:** compare speed and memory.
3. **Representation:** fixed buffers, fewer blocks.
4. **Inline storage:** `SmallVec` , `ArrayVec` .
5. **Replay history:** latency and misses per operation.

When the evidence justifies it

6. **Arenas:** `bumpalo` ; **API bridge:** `allocator-api2` .
7. **Allocator-aware collections:** `hashbrown` .
8. **Owned arena results:** `multitude` ; check retention and thread-safety.
9. Then, and only then, **write one yourself**.

Takeaways

1. **Understand how allocators work** - even if `Global` is the only allocator you ever use. Every `Vec`, every `Box` is a placement decision you delegated.
2. **Stop blaming the CPU: your heap is the crime scene.** Flat live data, rising p99, more cache misses per operation - investigate placement; those symptoms alone do not prove diffusion.
3. **Choosing the wrong strategy leaves performance on the table.** Adding an unnecessary boundary can lose too. Measure your workload, not the headline.

Don't stop at how fast your allocator is. Ask where it put your data.

Allocator-Aware Rust

NDC TechTown

Kongsberg - September 2026

Victor Ciura

Principal Architect
Rust Tooling @ Microsoft

Bonus Slides

The bits between the bits

The 6 indicators

- **F** = potential for a subsystem's memory to be diffused by *other* subsystems' allocations. Nonzero whenever other subsystems exist **and** this one allocates more than one chunk.
- **D** = (number of allocate/deallocate instructions) / (total instructions)
- **V** characterizes the distribution of requested sizes, not a standardized scalar score.
- **L**: for a subregion over a duration, with *I* access instructions, *M* bytes, *T* exits:
 - physical locality $L = I / M$ (accesses per byte)
 - temporal locality $L = I / T$ (accesses between exits)
- **U** = peak live allocated bytes / cumulative allocated bytes, for one subsystem and interval.
- **C** characterizes concurrent allocation/free pressure per thread; strongly correlated with **D**.

Fragmentability, in Rust types

<code>Vec<i32></code>	<code>// 1 allocation, contiguous</code>	→ LOW
<code>Box<[u8]></code>	<code>// 1 allocation, fixed length</code>	→ LOW
<code>String</code>	<code>// 1 allocation</code>	→ LOW
<code>Vec<Vec<i32>></code>	<code>// 1 + N allocations, N owners</code>	→ HIGH
<code>HashMap<String, Value></code>	<code>// table + key buffers + Value's payloads</code>	→ HIGH
<code>BTreeMap<K, V></code>	<code>// a node per ~B entries, grown over time</code>	→ HIGH
<code>Vec<Box<dyn Trait>></code>	<code>// 1 + N, each a separate global alloc</code>	→ HIGH

Every `Box` inside a collection is an invitation for someone else's data to sit between your nodes.

snmalloc: freeing is a message

- Classic problem: thread A allocates, thread B frees. Remote frees can require coordination between allocators.
- snmalloc: **B batches messages to A**. Freed memory returns via a queue, amortizing synchronization.
- Excellent fit for **producer/consumer** and pipeline architectures - which describes a lot of Rust.
- Available as `snmalloc-rs`.

Allocation flags, and Rust for Linux

```
// one proposed backwards-compatible extension
pub unsafe trait AllocatorWithFlags {
    type ExtraArgs;
    fn allocate(&self, layout: Layout, args: Self::ExtraArgs)
        → Result<NonNull<u8>>, AllocError>;
}

pub unsafe trait Allocator: AllocatorWithFlags<ExtraArgs = ()> { ... }
```

- The kernel needs allocator-specific flags: `GFP_KERNEL` , `GFP_ATOMIC` , and so on.
- Only on **allocate** - deallocation is unaffected.
- Deferred from the MVP; kernel-specific flags remain a **downstream concern**.

Pinning and cloning: where the API gets subtle

```
// If two allocator values are clones, may either free the other's allocations?  
let a = MyAlloc::new();  
let b = a.clone();  
let p = a.allocate(layout)?;           // NonNull<u8>  
unsafe { b.deallocate(p.cast(), layout) }; // sound? depends entirely on MyAlloc
```

- `std::sync::Arc<T, A>::clone` quietly **relied** on that being true - a requirement that was never properly documented.
- It cannot be an ordinary `Clone` bound: safe downstream code could implement `Clone` wrongly and cause **UB**. The guarantee lives on `AllocatorClone` (experimental, not MVP).
- An arena may reclaim on drop. `std::boxed::Box::into_pin` requires `StaticAllocator` on nightly; `Pin::new` for an `Unpin` target is a different case.

Cross-allocator relocation is not free

Moving an owner transfers its **handle**, not its backing storage. Keeping a destination region is a different operation.

Rust: allocator-api2

```
fn move_handle<T, A: Allocator>(  
    dst: &mut Vec<T, A>, src: Vec<T, A>,  
) { *dst = src; } // allocator moves too  
  
fn rehome<T, A: Allocator, B: Allocator>(  
    dst: &mut Vec<T, A>, src: Vec<T, B>,  
) {  
    dst.clear();  
    for v in src { dst.push(v); }  
}
```

C++ (when the resources differ)

```
// pmr does NOT propagate on move  
// assignment. If the resources are  
// unequal, this is NOT a handle move:  
// elements must move into dst's  
// storage. Element-wise work: O(N).  
dst = std::move(src);
```

Rust assignment transfers the allocator value too. Keeping the destination region requires explicit rebuilding.

Fix: fewer allocations, fewer pointers

Merge the three lists

Three `Box<[Record]>` (16 bytes each) become one list plus two `u16` section offsets.

-28 bytes per entry.

Packing booleans into a bitflag removed **padding** - the struct shrank by *more* than the fields themselves.

Drop the redundant owner

```
pub struct Record {  
    owner: Option<Box<Name>> ,  
    // ...  
}
```

Most owners equal the queried name - already in the cache key.

`None` = no heap allocation at all.